

Mice Protein Expression

Statistical Learning Master in Statistics for Data Science

Amara Scheel

Noon Elsheikh

Alberto Peces Contreras

2026-21-02

Contents

1	Introduction	2
1.1	Data description	2
2	Data Preparation	2
2.1	Auxiliary tools	2
2.2	Packages	3
2.3	Data Loading & Imputation	4
2.4	Multicollinearity	6
3	Exploratory Data Analysis (EDA)	8
3.1	Class Imbalance Assessment	8
3.2	Feature Distribution and Normality Assumption	9
3.3	Inter-group Protein Expression	11
3.4	PCA	14
4	Modeling Strategy	16
4.1	Auxiliary tools for classifiers	16
4.2	Feature Selection Pipeline	17
4.3	Model Training	23
5	Results & Evaluation	25
5.1	Robust Cross-Validation	25
5.2	Model Comparison (Test Set)	26
5.3	ROC Curves	27
5.4	Threshold Exploration	28
5.5	Threshold Exploration (Medical Context Tuning)	29

1 Introduction

1.1 Data description

Research Context: This project is based on the research article “*Self-Organizing Feature Maps Identify Proteins Critical to Learning in a Mouse Model of Down Syndrome.*”

Dataset Source: UCI Machine Learning Repository: Mice Protein Expression

Biological Background: Down syndrome (DS) is a chromosomal abnormality (trisomy of human chromosome 21) associated with intellectual disability. The overexpression of genes encoded by the extra chromosome is believed to perturb standard pathways, causing learning deficits.

Goal: We design a strategy based on probabilistic classifiers—Gaussian Naive Bayes (GNB), Linear Discriminant Analysis (LDA), and Quadratic Discriminant Analysis (QDA)—to identify biologically important differences in protein expression levels in mice.

We include Logistic Regression as a discriminative baseline model. While LDA/QDA model the joint distribution of the predictors (generative), Logistic Regression directly models the posterior probabilities, providing a robust, assumption-light benchmark against which to evaluate our Gaussian models.

Structure: The dataset comprises expression levels of 77 proteins measured across 1007 tissue samples. The samples are divided into two main Genotypes: * **Control:** Normal genotype mice. * **Ts65Dn:** Trisomic (Down Syndrome) mice.

Additional factors include Treatment (Saline vs. Memantine) and Behavior (Stimulated vs. Not Stimulated). For this midterm project, our primary target variable is the **Genotype** (*Control* vs. *Ts65Dn*).

2 Data Preparation

2.1 Auxiliary tools

We define helper functions to automate repetitive tasks: `checkNA` for visualizing missing data, `imputation` for filling missing values, and `SelectKBest` for filtering features using ANOVA.

```
checkNA <- function(data){
  # For every column, we count the number of NA
  na_counts = colSums(is.na(data))

  # Default return if no NAs
  if(sum(na_counts) == 0) {
    return(character(0))
  }

  # Compute (N° columns with NA) / (N° columns)
  if (sum(na_counts > 0) > 0){
    p = round(length(names(na_counts[na_counts > 0])) / ncol(data), 2)
    na_prop = sort(na_counts[na_counts > 0] / nrow(data), decreasing = TRUE)
  } else {
    p = 0
    na_prop = numeric(0)
  }

  # BARPLOT (Only plot if there are actually NAs to show)
  if(length(na_prop) > 0) {
    barplot(na_prop, las = 2, cex.names = 0.6, ylab = "Proportion of NA values",
            main = "Missing-value proportion across features", col = "gray")
  }
}
```

```

abline(h = 0.05, col = "red", lty = 2)
legend("topright",
      legend = c(paste0("Features with NA values: ", p*100, "%"),
                  "Threshold for high missingness"),
      col = c("gray", "red"), pch = c(15, NA), lty = c(NA, 2), bty = "n")
grid()
}

# Return names of columns with > 5% missing data
return (names(na_prop[na_prop > 0.05]))
}

imputation <- function(df, columns, method = "median"){
  # For every column, we compute the median or mean value
  for (col_j in columns){
    if (method == "median"){
      val = median(df[[col_j]], na.rm = TRUE)
    }else{val = mean(df[[col_j]], na.rm = TRUE)}
    # Replace the NA value with median or mean
    df[[col_j]][is.na(df[[col_j]])] = val
  }
  return(df)
}

SelectKBest <- function(X, y, K = 10){
  # Define auxiliary parameters
  n_features = ncol(X)
  f_score = numeric(n_features)
  names(f_score) = colnames(X)

  # For each feature, we compute ANOVA test between groups
  for (j in 1:n_features){
    # Safe extraction of F-value to prevent crash
    tryCatch({
      fit <- aov(X[,j] ~ y)
      summ <- summary(fit)
      if(!is.null(summ[[1]][["F value"]])) {
        f_score[j] = summ[[1]][["F value"]][1]
      } else {
        f_score[j] = 0
      }
    }, error = function(e) { f_score[j] = 0 })
  }

  # We sort the F scores in decreasing order and select the best K features
  K = min(K, n_features)
  top_K = names(sort(f_score, decreasing = TRUE))[1:K]
  return (top_K)
}

```

2.2 Packages

```

library(tidyverse)
library(ggplot2)
library(caret)

```

```
library(class)
library(pROC)
library(e1071) # For Naive Bayes
library(MASS)  # For LDA/QDA
library(klaR)  # Additional tools
```

2.3 Data Loading & Imputation

The first step is to identify missing values. When handling missing data, it is important to consider the underlying mechanism: Missing Completely at Random (MCAR), Missing at Random (MAR), or Missing Not at Random (MNAR).

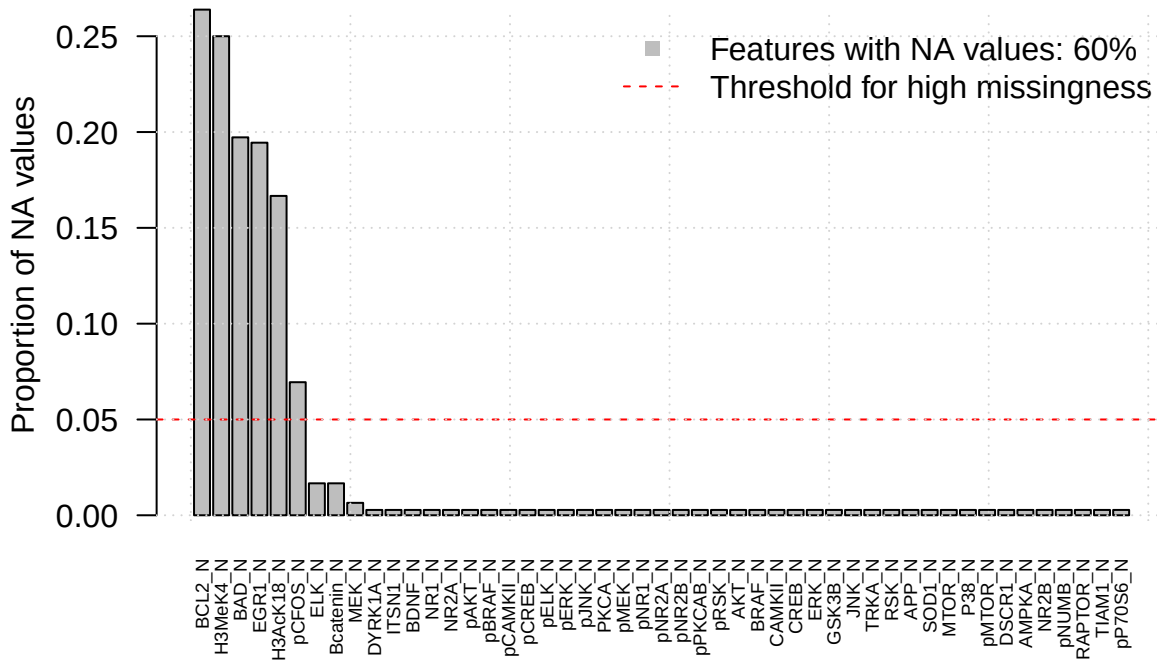
In proteomic datasets, high rates of missingness are frequently MNAR, occurring when protein expression is too low for detection. Applying median imputation to MNAR data would artificially inflate the unobserved low values up to the group median, creating systemic bias.

To mitigate this risk, we adopt a threshold: * **Removal (>5% missingness)**: Features with a high proportion of missing data are assumed to carry a high risk of being MNAR and are excluded entirely from subsequent analyses to prevent imputation-induced bias. * **Imputation (<5% missingness)**: For variables with minimal missingness, we assume the absence is largely due to technical noise or measurement error (MCAR/MAR). To preserve these features without dropping valuable patient rows, we apply median imputation.

```
# -----
# Load data
# -----

### separator may be ; or , depending on the dataset !!!
data = read.csv("Data_Cortex_Nuclear.csv", sep = ",", header = TRUE)
colNames = colnames(data)
# -----
# Different labels
# -----
metadata = c("MouseID", "Genotype", "Treatment", "Behavior", "class")
# -----
# Missing-value proportion across features
# -----
cols_with_NA = checkNA(data)
```

Missing-value proportion across features



```
proteins = colNames[!(colNames %in% c(metadata, cols_with_NA))]
```

Imputation Strategy: To prevent data leakage, we first perform a stratified 70/30 train/test split before any imputation takes place. We then divide the training set into 4 biological subgroups (Control/Ts65Dn × Memantine/Saline) and compute the median protein expression within each group exclusively from the training data. These medians are subsequently applied to impute missing values in both the training and test sets. This ensures two things: first, that the characteristics of treated mice do not artificially influence control mice imputation values; and second, that the test set is imputed using only information available at training time, faithfully simulating a real deployment scenario where new observations arrive without known labels.

```
# 1. Split BEFORE imputation
set.seed(619)
lvls <- c("Control", "Ts65Dn")
classes <- factor(data$Genotype, levels = lvls)

fltr_cols <- !(colNames %in% cols_with_NA)
data_clean <- data[, fltr_cols]

idx_train <- createDataPartition(classes, p = 0.7, list = FALSE)
data_train <- data_clean[idx_train, ]
data_test <- data_clean[-idx_train, ]

# 2. Compute subgroup medians from TRAINING SET only
get_group_medians <- function(df, proteins) {
  df %>%
    group_by(Genotype, Treatment) %>%
    summarise(across(all_of(proteins),
```

```

      \ (x) median(x, na.rm = TRUE)), .groups = "drop")
}

train_medians <- get_group_medians(data_train, proteins)

# 3. Impute using pre-computed training medians
impute_with_medians <- function(df, medians, proteins) {
  for (col in proteins) {
    na_idx <- which(is.na(df[[col]]))
    for (i in na_idx) {
      ref_val <- medians %>%
        filter(Genotype == df$Genotype[i],
               Treatment == df$Treatment[i]) %>%
        pull(col)
      df[[col]][i] <- ref_val
    }
  }
  return(df)
}

data_train <- impute_with_medians(data_train, train_medians, proteins)
data_test  <- impute_with_medians(data_test,  train_medians, proteins)

# 4. Reconstruct data_filtered to preserve compatibility with downstream code
data_filtered <- rbind(data_train, data_test)

# 5. Verify no NAs remain
cols_for_removal <- checkNA(data_filtered)

# 6. Extract X_train, X_test
X_train <- data_train[, proteins]
X_test  <- data_test[, proteins]

```

2.4 Multicollinearity

LDA and QDA require invertible covariance matrices; high multicollinearity renders these singular or unstable. We empirically determined that thresholds above 0.70 caused QDA to fail due to ill-conditioned class covariance matrices, so we fix our cutoff at 0.70. Features are removed using `findCorrelation` from `caret`, which iteratively drops only the single most correlated variable from each pair, preserving maximum predictive information.

```

# Compute correlation matrix
cor_mat <- cor(data_filtered[,proteins])
cor_df <- as.data.frame(as.table(cor_mat))
colnames(cor_df) <- c("X1", "X2", "Correlation")

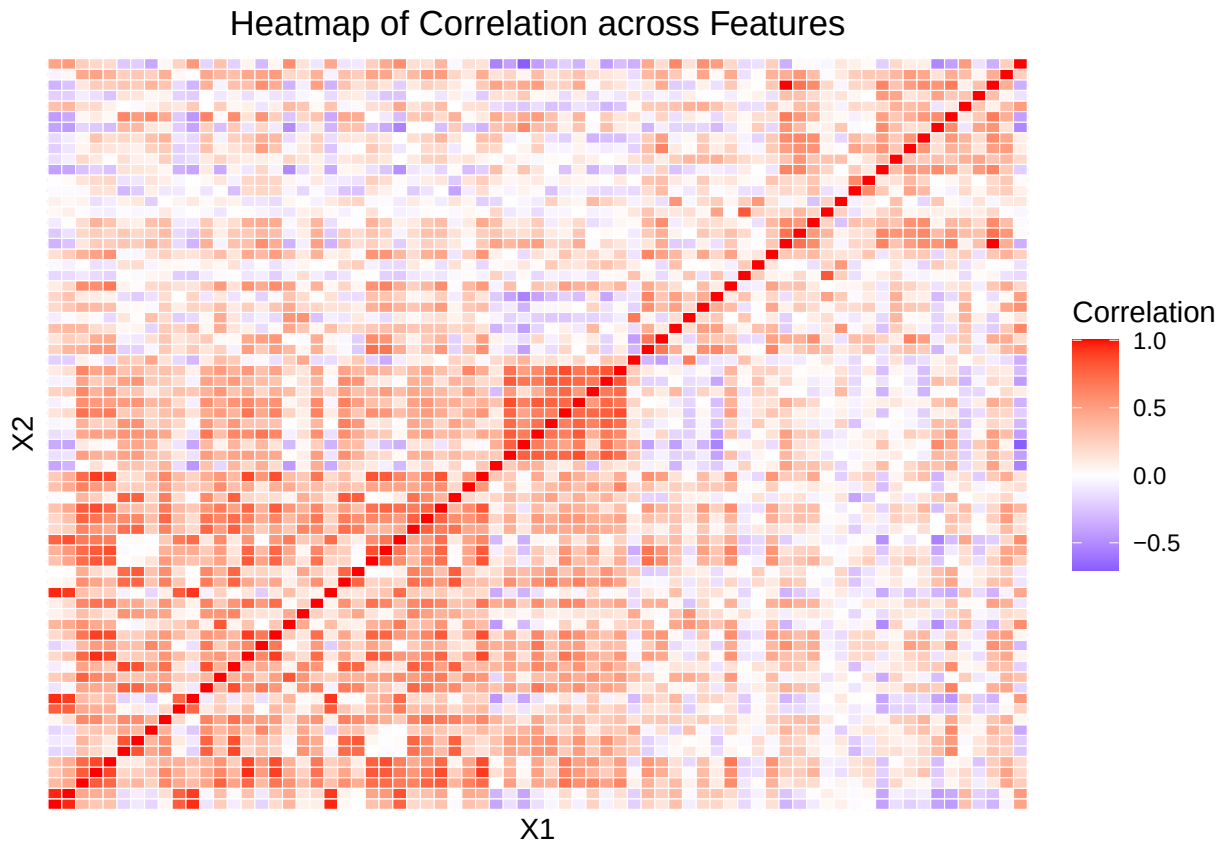
# Heatmap plot
ggplot(cor_df, aes(x = X1, y = X2, fill = Correlation)) +
  geom_tile(color = "white") +
  scale_fill_gradient2(low = "blue", mid = "white", high = "red", midpoint = 0) +
  theme_minimal() +
  theme(
    axis.text.x = element_blank(),

```

```

axis.text.y = element_blank(),
axis.ticks.y = element_blank(),
plot.title = element_text(hjust = 0.5)
) +
ggtitle("Heatmap of Correlation across Features")

```



```

# Identify correlated features to remove (>0.7)

# We use caret::findCorrelation which removes only ONE
# feature from a highly correlated pair (the one with the highest
# average correlation to all other features).

cut_off = round(seq(0.0, 1.0, length.out = 25), 2)
num_features = numeric(length(cut_off))

# Visualize impact of thresholds
for (i in seq_along(cut_off)){
  # Find how many features would be removed at each cutoff
  to_remove = findCorrelation(cor_mat, cutoff = cut_off[i])
  num_features[i] = length(to_remove)
}

# Apply the chosen 0.70 threshold
high_cor_idx = findCorrelation(cor_mat, cutoff = 0.7, names = TRUE)
n_high_cor = length(high_cor_idx)

# Select uncorrelated proteins

```

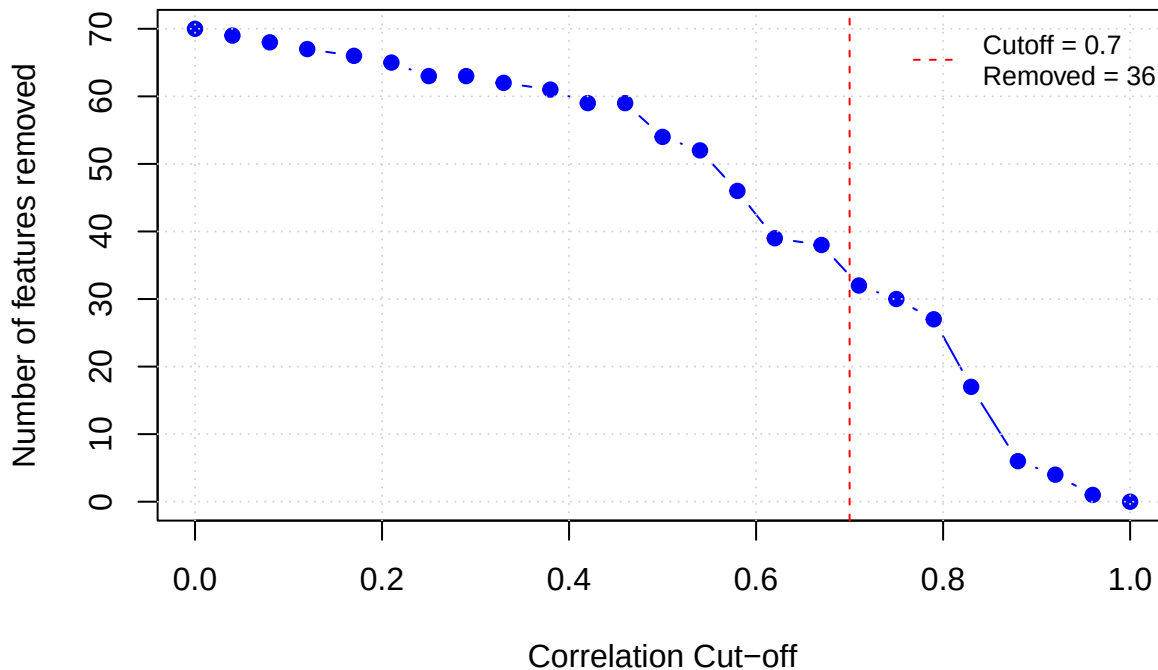
```

# Remove the identified highly correlated features from our protein list
if(length(high_cor_idx) > 0) {
  proteins = colNames[!(colNames %in% c(metadata, cols_with_NA, high_cor_idx))]
} else {
  proteins = colNames[!(colNames %in% c(metadata, cols_with_NA))]
}

# Plot: Number of variables removed vs. cut-off
plot(cut_off, num_features, type = "b", pch = 19, col = "blue",
     xlab = "Correlation Cut-off", ylab = "Number of features removed",
     main = "Features Removed vs. Correlation Cut-off")
abline(v = 0.7, col = "red", lty = 2)
grid()
legend("topright", legend = paste("Cutoff = 0.7\nRemoved = ", n_high_cor),
      col = "red", lty = 2, cex = 0.8, bty = "n")

```

Features Removed vs. Correlation Cut-off



3 Exploratory Data Analysis (EDA)

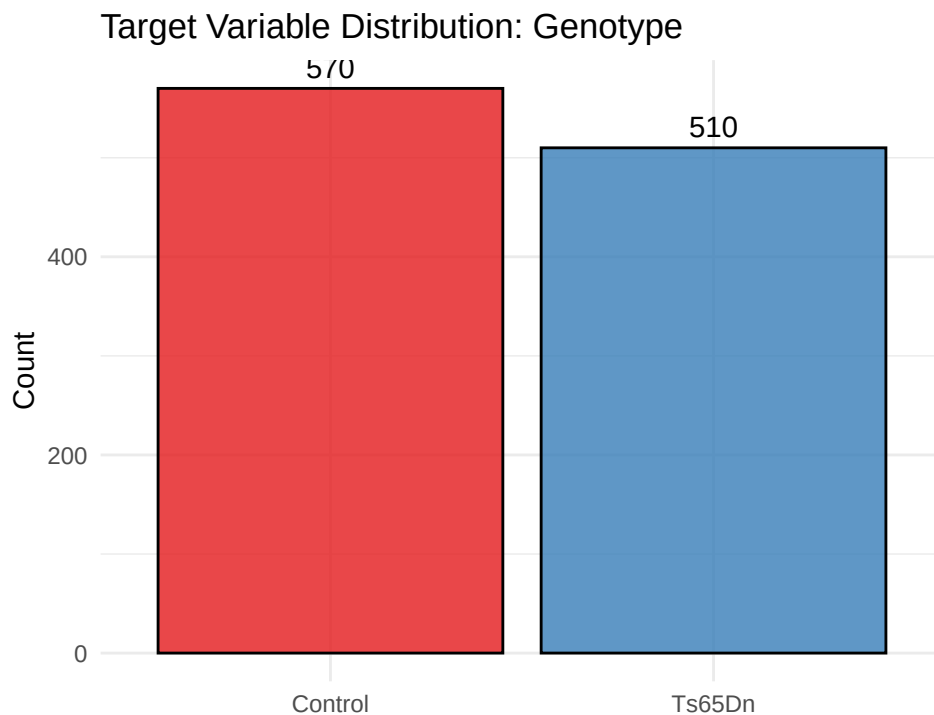
Before modeling, we must thoroughly investigate the dataset's target distribution, the underlying shapes of the feature distributions (as our chosen probabilistic models heavily rely on normality assumptions), and the multivariate expression profiles across classes.

3.1 Class Imbalance Assessment

First, we examine the distribution of our target variable, **Genotype**. Significant class imbalance can skew model training and render standard accuracy metrics misleading.


```
# Calculate proportions
class_counts <- table(data_filtered$Genotype)
class_props <- prop.table(class_counts)

ggplot(data_filtered, aes(x = Genotype, fill = Genotype)) +
  geom_bar(alpha = 0.8, color = "black") +
  geom_text(stat='count', aes(label=..count..), vjust=-0.5) +
  theme_minimal() +
  labs(title = "Target Variable Distribution: Genotype",
       y = "Count", x = "") +
  scale_fill_brewer(palette = "Set1") +
  theme(legend.position = "none")
```



```
print(round(class_props, 3))
```

```
##
## Control  Ts65Dn
##  0.528   0.472
```

Interpretation: The bars for Control and Ts65Dn are roughly the same height. This means Accuracy is a reliable metric, and the model has enough examples of both classes to learn fairly.

3.2 Feature Distribution and Normality Assumption

GNB, LDA, and QDA inherently assume that the continuous features follow a Gaussian (Normal) distribution within each class. To validate this assumption, we identify the single most discriminative protein using an ANOVA F-test and examine its distribution via Density and Q-Q (Quantile-Quantile) plots.

```

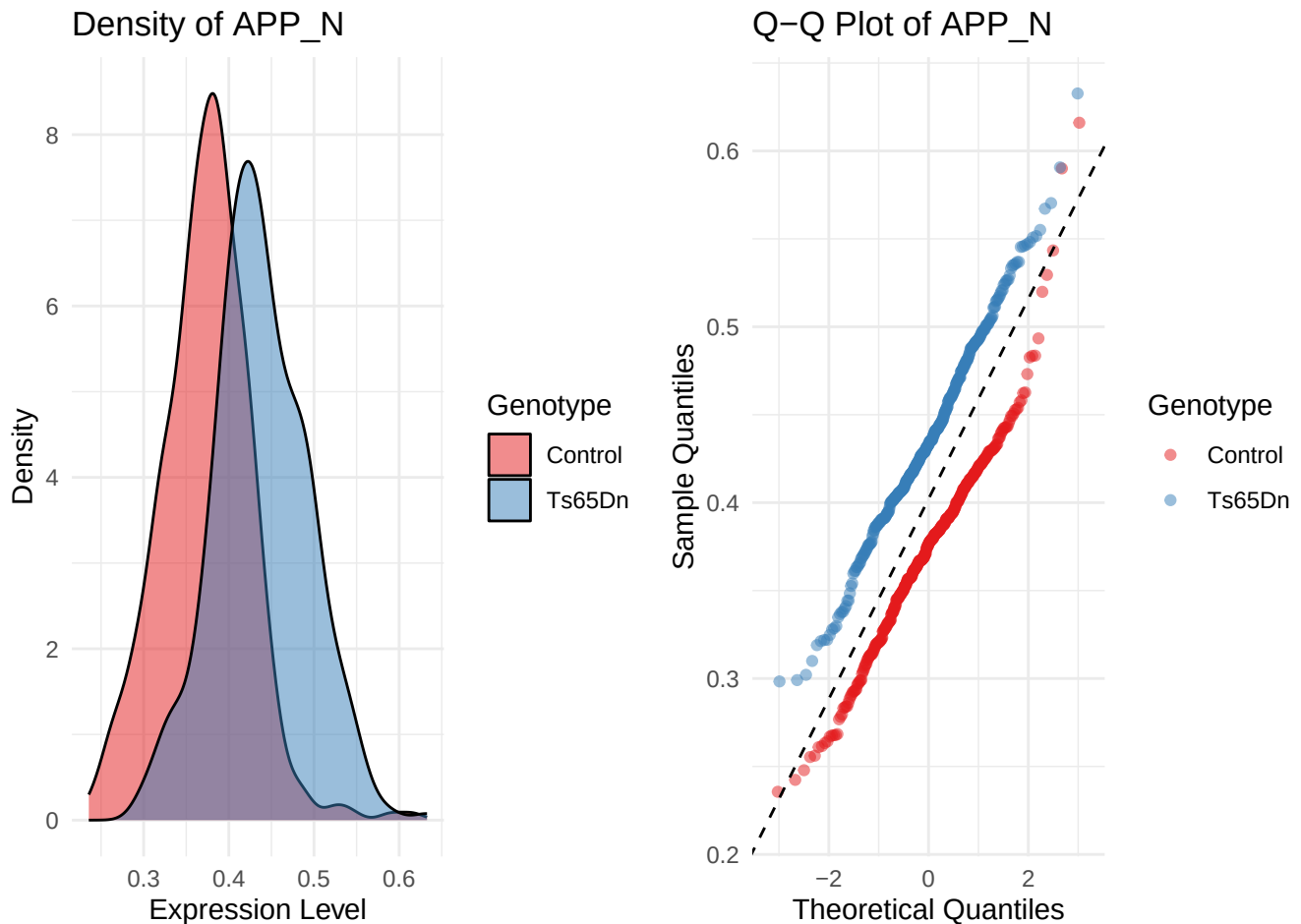
# 1. Use SelectKBest to find the #1 best feature
top_feature_name <- SelectKBest(data_train[, proteins], data_train$Genotype, K = 1)

# 2. Density Plot
p1 <- ggplot(data_train, aes(x = .data[[top_feature_name]], fill = Genotype)) +
  geom_density(alpha = 0.5) +
  theme_minimal() +
  labs(title = paste("Density of", top_feature_name),
       x = "Expression Level", y = "Density") +
  scale_fill_brewer(palette = "Set1")

# 3. Q-Q Plot
p2 <- ggplot(data_train, aes(sample = .data[[top_feature_name]], color = Genotype)) +
  stat_qq(alpha = 0.5) + stat_qq_line(color = "black", linetype = "dashed") +
  theme_minimal() +
  labs(title = paste("Q-Q Plot of", top_feature_name),
       x = "Theoretical Quantiles", y = "Sample Quantiles") +
  scale_color_brewer(palette = "Set1")

# Display side by side
gridExtra::grid.arrange(p1, p2, ncol = 2)

```



Interpretation: The density plot shows a clear separation between the group means, confirming its predictive power. The Q-Q plot shows that the quantiles of the data generally follow the theoretical diagonal line of a normal

distribution. While there are slight deviations in the extreme tails, the overall Gaussian assumption holds sufficiently well to proceed with LDA and QDA.

3.3 Inter-group Protein Expression

To move beyond univariate analysis, we extract the top 10 most discriminative proteins and visualize their average expression profiles across the healthy and trisomic classes.

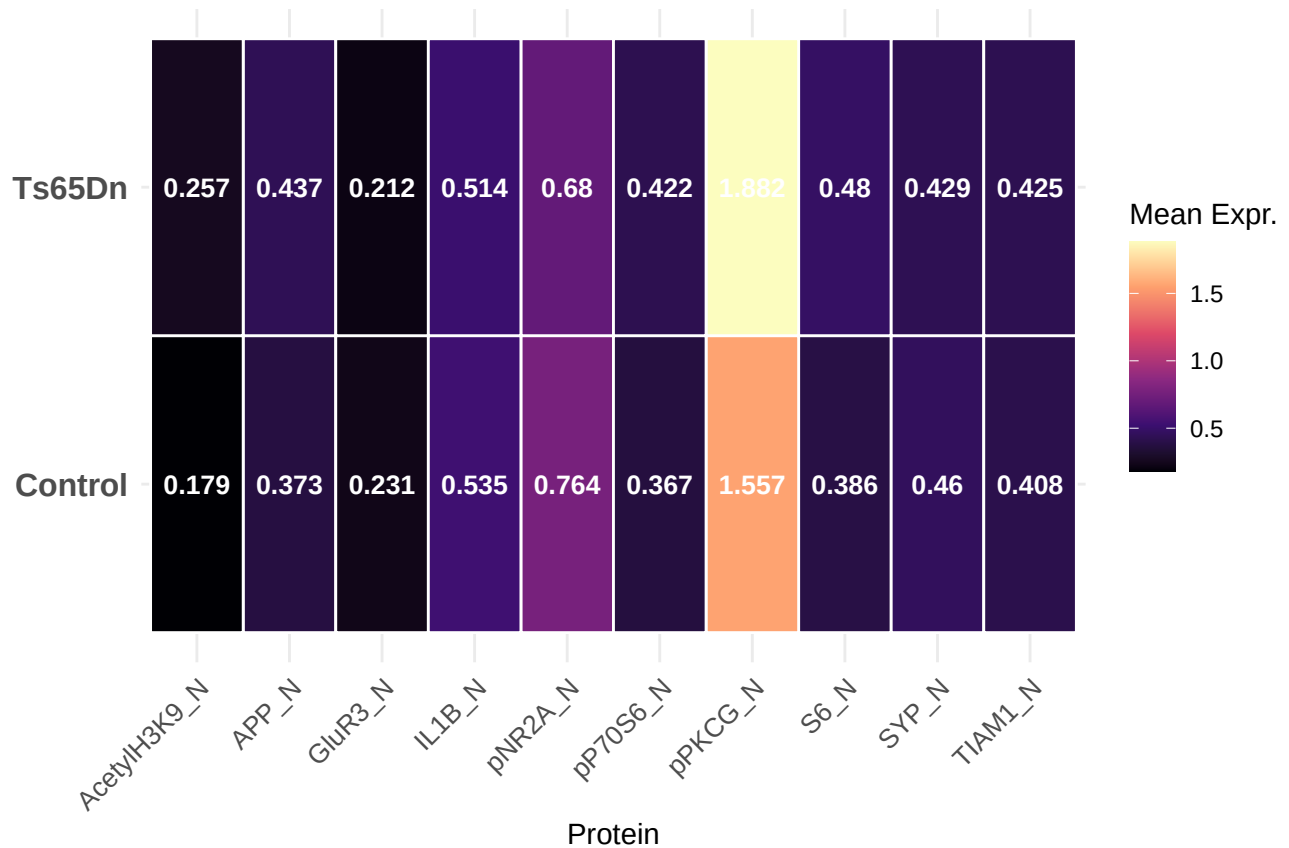
```
# Get top 10 features
top_10_features <- SelectKBest(data_train[, proteins], data_train$Genotype, K = 10)

# Compute mean expression of these top 10 proteins per Genotype
heatmap_data <- data_train %>%
  group_by(Genotype) %>%
  summarise(across(all_of(top_10_features), mean, na.rm = TRUE)) %>%
  pivot_longer(cols = -Genotype, names_to = "Protein", values_to = "Mean_Expression")

# Plot Heatmap
ggplot(heatmap_data, aes(x = Protein, y = Genotype, fill = Mean_Expression)) +
  geom_tile(color = "white", size = 0.5) +
  geom_text(aes(label = round(Mean_Expression, 3)), # <-- prints values on tiles
    color = "white", size = 3.5, fontface = "bold") +
  scale_fill_viridis_c(option = "magma") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1, size = 10),
    axis.text.y = element_text(size = 12, face = "bold")) +
  labs(title = "Expression Profiles of Top 10 Predictor Proteins",
    subtitle = "Average expression levels grouped by Genotype",
    x = "Protein", y = "", fill = "Mean Expr.")
```

Expression Profiles of Top 10 Predictor Proteins

Average expression levels grouped by Genotype



```
effect_sizes <- data_train %>%
  summarise(across(all_of(top_10_features), ~ {
    ctrl <- .x[data_train$Genotype == "Control"]
    ts <- .x[data_train$Genotype == "Ts65Dn"]
    pooled_sd <- sqrt((var(ctrl) + var(ts)) / 2)
    (mean(ts) - mean(ctrl)) / pooled_sd # Cohen's d
  })) %>%
  pivot_longer(everything(), names_to = "Protein", values_to = "Cohens_d")

cat("Effect Sizes (Cohen's d) per Protein\n")
```

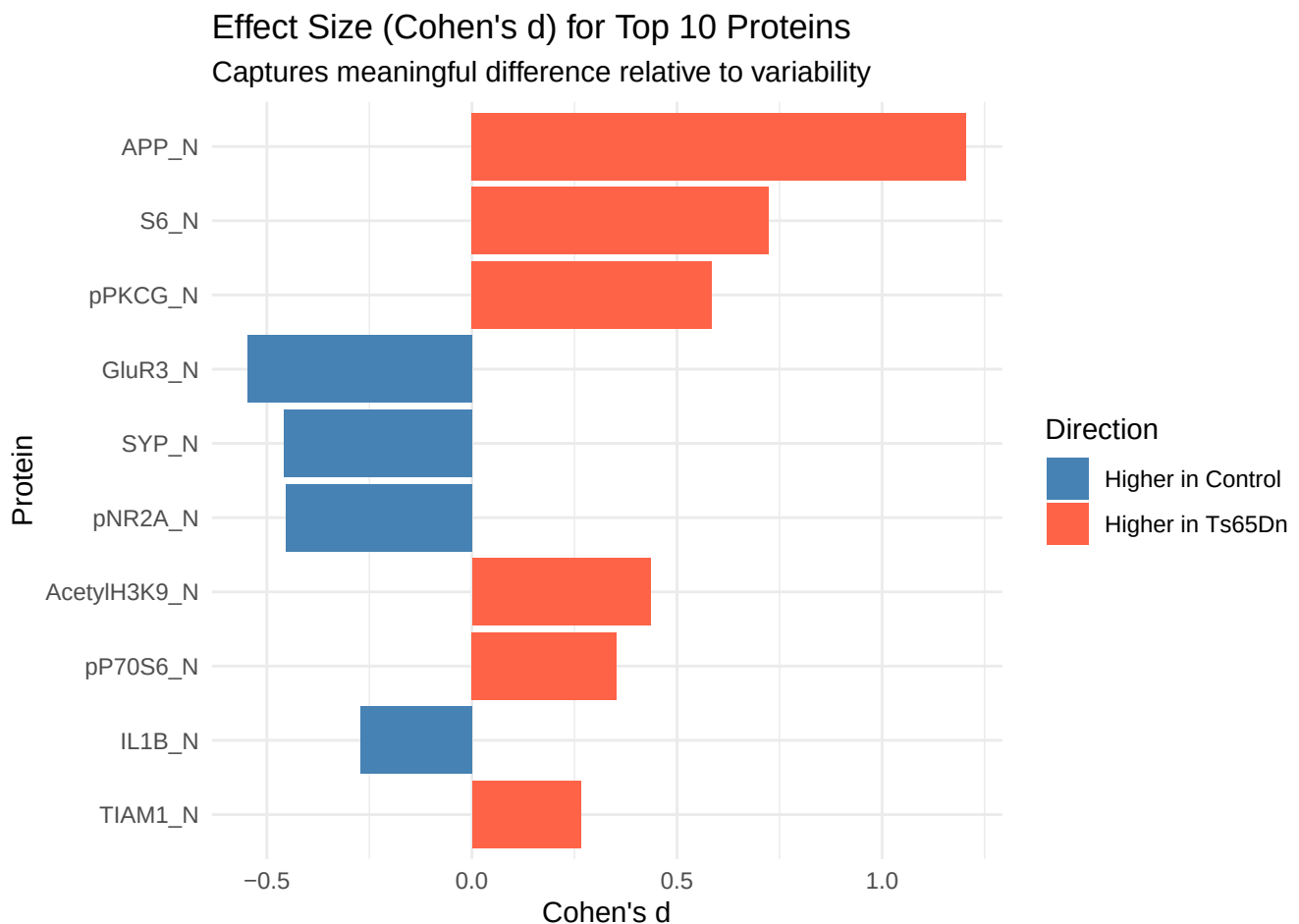
```
## Effect Sizes (Cohen's d) per Protein
```

```
print(effect_sizes %>% arrange(desc(abs(Cohens_d))), digits = 3)
```

```
## # A tibble: 10 x 2
##   Protein      Cohens_d
##   <chr>         <dbl>
## 1 APP_N         1.21
## 2 S6_N          0.724
## 3 pPKCG_N       0.584
## 4 GluR3_N      -0.546
## 5 SYP_N        -0.459
## 6 pNR2A_N      -0.452
```

```
## 7 AcetylH3K9_N    0.435
## 8 pP70S6_N        0.352
## 9 IL1B_N          -0.273
## 10 TIAM1_N         0.265
```

```
ggplot(effect_sizes, aes(x = reorder(Protein, abs(Cohens_d)),
                           y = Cohens_d,
                           fill = Cohens_d > 0)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  scale_fill_manual(values = c("steelblue", "tomato"),
                    labels = c("Higher in Control", "Higher in Ts65Dn")) +
  theme_minimal() +
  labs(title = "Effect Size (Cohen's d) for Top 10 Proteins",
       subtitle = "Captures meaningful difference relative to variability",
       x = "Protein", y = "Cohen's d", fill = "Direction")
```



Interpretation: Among the top 10 proteins, raw mean expression differences between genotypes are small (typically 0.05–0.10), which is expected given that protein expression data is already normalized — small absolute differences do not mean the proteins are unimportant.

Effect size analysis shows that two proteins have large effects ($d > 0.8$, one above $d = 1.0$), meaning the difference between groups is larger than the natural variation within groups which is a strong signal. Four proteins show medium effects ($d = 0.5$), and the remaining three are weaker at $d = 0.3$. Overall, classification appears to be driven by a small set of highly discriminant proteins, with the rest providing supporting information.

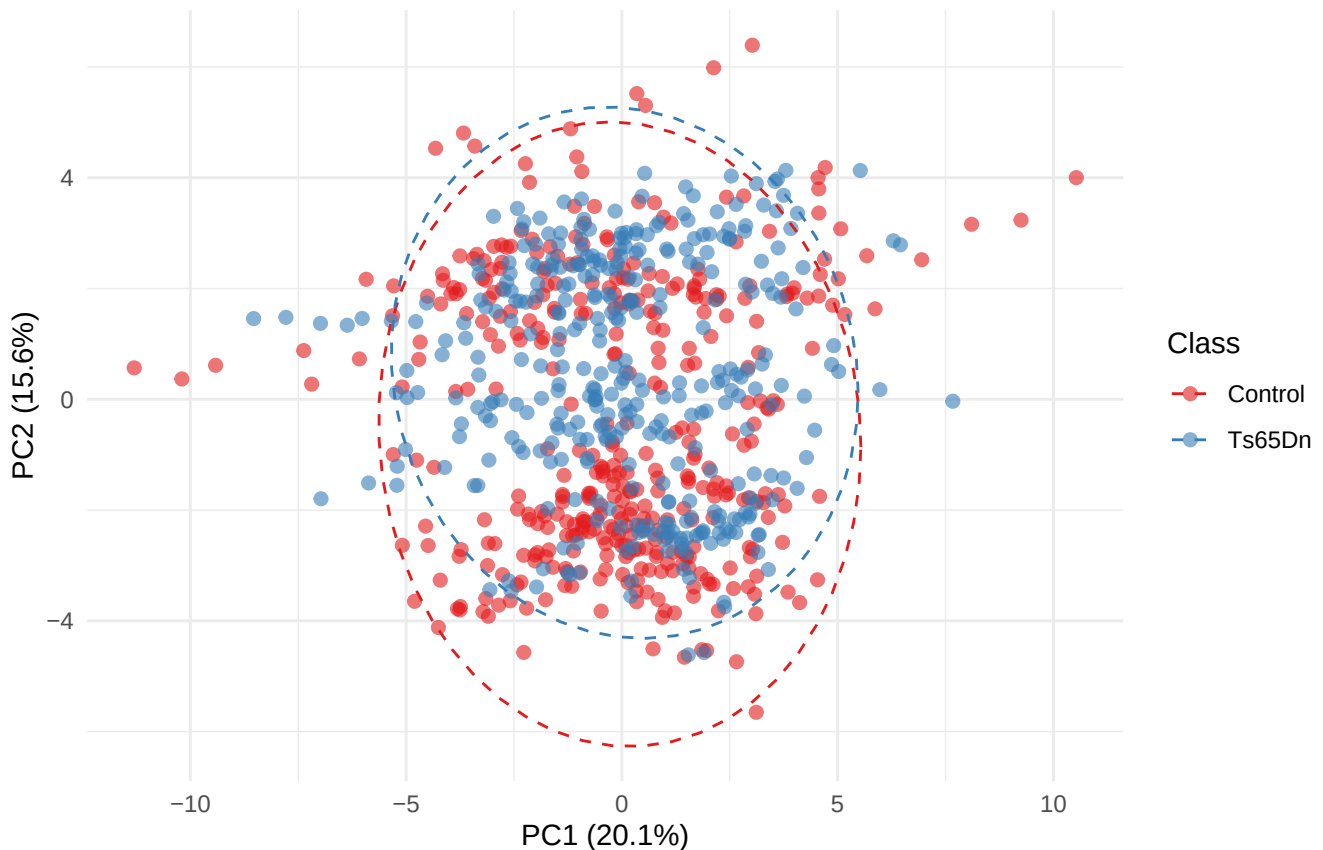
3.4 PCA

Finally, we perform Principal Component Analysis (PCA) utilizing the entire filtered protein set to evaluate global class separability in lower dimensions.

```
# Compute PCA on the protein data
# Compute PCA on the protein data
pca_res <- prcomp(data_train[, proteins], scale. = TRUE)
pca_df <- as.data.frame(pca_res$x)
pca_df$Class <- data_train$Genotype

ggplot(pca_df, aes(x = PC1, y = PC2, color = Class)) +
  geom_point(alpha = 0.6, size = 2) +
  stat_ellipse(level = 0.95, linetype = 2) +
  theme_minimal() +
  labs(title = "PCA: First Two Principal Components",
       subtitle = "Visualizing Class Separation",
       x = paste0("PC1 (", round(summary(pca_res)$importance[2,1]*100, 1), "%)"),
       y = paste0("PC2 (", round(summary(pca_res)$importance[2,2]*100, 1), "%)")) +
  scale_color_brewer(palette = "Set1")
```

PCA: First Two Principal Components
Visualizing Class Separation

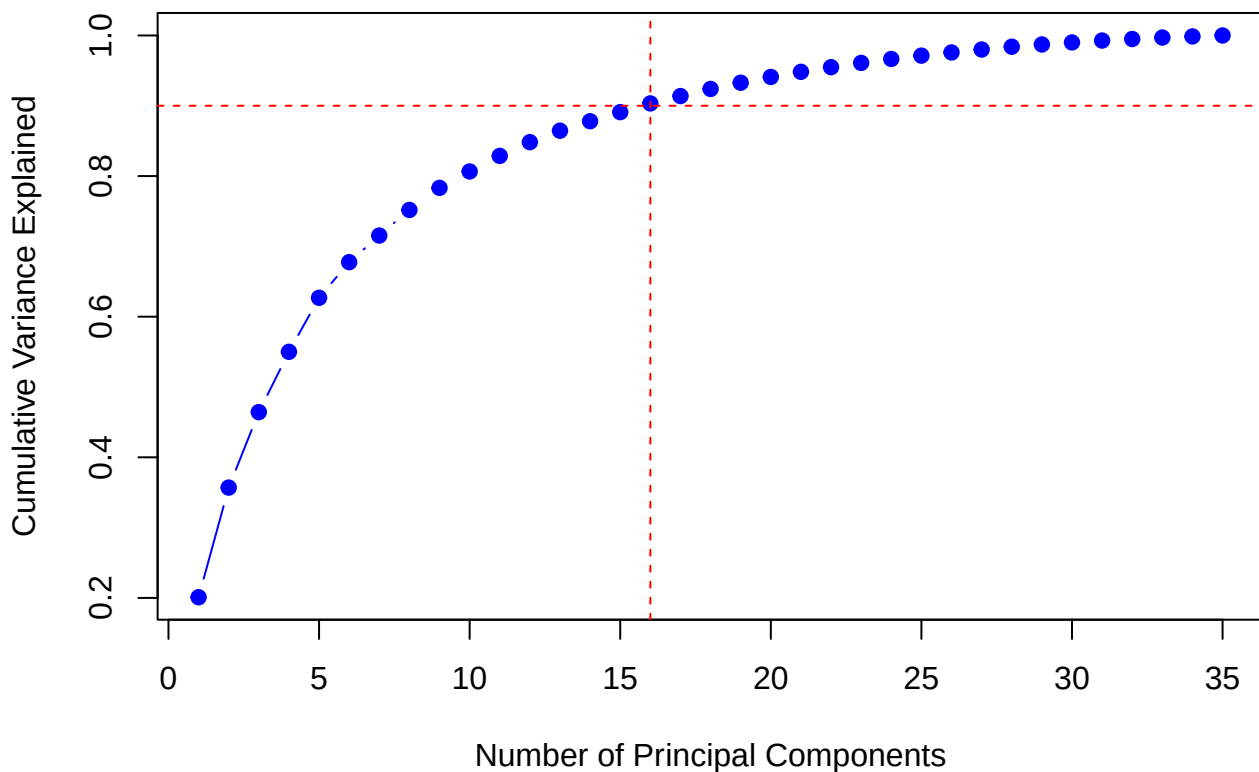


Interpretation of PCA: The first two principal components explain approximately 36% of the total variance. The visualization shows a significant overlap between the *Control* and *Ts65Dn* classes, indicating that they are not linearly separable in these lower dimensions.

```
# Cumulative variance explained
var_explained <- summary(pca_res)$importance[3, ] # cumulative proportion
n_components <- which(var_explained >= 0.90)[1]    # components for 90%

plot(var_explained, type = "b", pch = 19, col = "blue",
     xlab = "Number of Principal Components",
     ylab = "Cumulative Variance Explained",
     main = "Scree Plot: Cumulative Variance Explained")
abline(h = 0.90, col = "red", lty = 2)
abline(v = n_components, col = "red", lty = 2)
```

Scree Plot: Cumulative Variance Explained



```
n_80 <- which(var_explained >= 0.80)[1] # components for 80%
n_90 <- which(var_explained >= 0.90)[1] # components for 90%

# Print statements
cat(sprintf("80%% of variance is explained by %d PCs\n", n_80))
```

```
## 80% of variance is explained by 10 PCs
```

```
cat(sprintf("90%% of variance is explained by %d PCs\n", n_90))
```

```
## 90% of variance is explained by 16 PCs
```

Interpretation of the Scree Plot: The dataset is highly multi-dimensional, to explain 80% to 90% of the variance we require 10 to 17 principal components, respectively. This explains the overlap in the 2D PCA plot and confirms that the classes are not linearly separable in low dimensions. It justifies our use of multivariate probabilistic models (LDA, QDA).

4 Modeling Strategy

4.1 Auxiliary tools for classifiers

We define some functions for each classifier, improving the future pipeline structure.

```
# -----
# Logistic Regression
# -----
LogReg_classifier <- function(df_train, df_test, y_test){

  # Build the LogReg model  $\log(\mu) = \beta_0 + X_1\beta_1 + \dots + X_n\beta_n$ 
  model = glm(y ~ ., data = df_train, family = binomial(link = "logit"))

  # Compute the probability corresponding to each observation
  probs = predict(model, newdata = df_test, type = "response")

  # Calculate the classes manually
  y_pred = factor(ifelse(probs > 0.5, "Ts65Dn", "Control"),
                  levels = c("Control", "Ts65Dn"))

  # Compute the accuracy
  acc = mean(y_pred == y_test)
  return(acc)
}

# -----
# Gaussian Naive Bayes
# -----
GNB_classifier <- function(df_train, df_test, y_test){

  # Build the Gaussian Naive Bayes model
  model = naiveBayes(y ~ ., data = df_train)

  # Calculate the predicted classes
  y_pred = predict(model, df_test, type = "class")

  # Compute the accuracy
  acc = mean(y_pred == y_test)
  return(acc)
}

# -----
# Linear Discriminant Analysis
# -----
LDA_classifier <- function(df_train, df_test, y_test){

  # Build the Linear Discriminant Analysis model
  model = lda(y ~ ., data = df_train)

  # Calculate the predicted classes
  pred = predict(model, df_test)
  y_pred = pred$class
}
```



```

# Compute the accuracy
acc = mean(y_pred == y_test)
return(acc)
}
# -----
# Quadratic Discriminant Analysis
# -----
QDA_classifier <- function(df_train, df_test, y_test){
  tryCatch({
    model = qda(y ~ ., data = df_train)
    pred = predict(model, df_test)
    y_pred = pred$class
    acc = mean(y_pred == y_test)
    return(acc)
  }, error = function(e){
    message("QDA failed (rank deficiency): returning NA")
    return(NA)
  })
}

```

4.2 Feature Selection Pipeline

The pipeline begins by preprocessing and feature selection to prepare the data for classification.

Z-score scaling:

- From the training dataset, the mean (μ) and standard deviation (σ) of each feature are computed.
- Both the training and testing sets are then standardized using these parameters.
- This ensures that all features contribute equally to the classifiers, avoiding bias from features with larger magnitudes.

SelectKBest with ANOVA:

- The pipeline selects the **top K features** that have the strongest univariate relationship with the target class, based on the ANOVA F-score.
- The value of K is defined before training and will be varied to determine the optimal number of features for each classifier.

Classification:

- Each classifier (LogReg, GNB, LDA, and QDA) is trained using only the selected **top-K features**.
- The pipeline computes the **average accuracy** on the test set. These accuracies are stored for later comparisons between classifiers allowing us to identify the value of K that gives the best performance.

```

# -----
# Function for pipeline workflow
# -----
run_pipeline <- function(X_train, X_test, y_train, y_test, topk = 10){

  # 1. Extract parameters for Z-score scaling
  mu = apply(X_train, 2, mean)

```

```

sigma = apply(X_train, 2, sd)

# 2. Scale train and test partitions with TRAIN parameters
X_train_sc = scale(X_train, center = mu, scale = sigma)
X_test_sc = scale(X_test, center = mu, scale = sigma)

# 3. Select K Best
topk = min(topk, ncol(X_train_sc)) # "topk" > ncol: not allowed
top_k = SelectKBest(X = X_train_sc, y = y_train, K = topk)

# 4. Select K features and turn matrix into dataframe format
X_train_df = data.frame(y = y_train, X_train_sc[,top_k, drop = FALSE])
X_test_df = data.frame(X_test_sc[,top_k, drop = FALSE])

# 5. LogReg classification
acc_LogReg = LogReg_classifier(X_train_df, X_test_df, y_test)

# 6. GNB classification
acc_GNB = GNB_classifier(X_train_df, X_test_df, y_test)

# 7. LDA classification
acc_LDA = LDA_classifier(X_train_df, X_test_df, y_test)

# 8. QDA classification
acc_QDA = QDA_classifier(X_train_df, X_test_df, y_test)

# Results
return(list(LogReg=acc_LogReg,GNB=acc_GNB,LDA=acc_LDA,QDA=acc_QDA,features=top_k))
}

```

The previous pipeline is extended to **5-Fold Cross-Validation** to obtain a more robust estimate of performance for each number of top features (topk).

Fold creation and initialization:

- The dataset is split into **5 folds**, and some variables are defined to **store accuracies** and the selected **feature subsets** for each classifier and fold.

Training and testing per fold

- For each fold, one subset, one subset is used as the test set, while the remaining 4 folds form the training set.
- This ensures that every data point is used once for testing and 4 times for training. In addition, each classifier will be trained using the same fold assignments to **avoid data leakage** and to ensure **generalization**.

Pipeline execution and data collection

- The previously pipeline (scaling, feature selection, classifier training) is applied to the training and testing sets in each fold.
- **Accuracy scores** and the **selected top-K features** are recorded for each classifier.
- After all folds, the **average performance** is computed to identify the optimal number of features and compare classifiers.

```

# Structure for K-Fold CV Wrapper
kfold_cv <- function(X, y, k = 5, topk = 10){
  folds = createFolds(y, k = k, list = TRUE, returnTrain = TRUE)
  features_list = vector("list", k)

  # Initialize with NA so failed folds don't count as "0% accuracy"
  scores = matrix(NA, nrow = 4, ncol = k,
                  dimnames = list(c("LogReg", "GNB", "LDA", "QDA"), NULL))

  for (i in seq_along(folds)){
    idx_train = folds[[i]]
    X_train_fold = X[idx_train,]
    y_train_fold = y[idx_train]
    X_test_fold = X[-idx_train,]
    y_test_fold = y[-idx_train]

    res = run_pipeline(X_train_fold, X_test_fold, y_train_fold, y_test_fold, topk)
    scores[, i] = c(res$LogReg, res$GNB, res$LDA, res$QDA)
    features_list[[i]] = res$features
  }

  # -----
  # STABILITY SELECTION to avoid selection bias
  # -----
  # 1. Unlist all features chosen across the folds
  all_selected <- unlist(features_list)

  # 2. Count the frequency of each feature (e.g., how many folds was it in?)
  feature_counts <- table(all_selected)

  # 3. Sort them by most frequent to least frequent, and pick the top K
  sorted_features <- sort(feature_counts, decreasing = TRUE)
  final_features <- names(sorted_features)[1:topk]

  out = list(
    cv_scores = scores,
    feat_LogReg = final_features,
    feat_GNB = final_features,
    feat_LDA = final_features,
    feat_QDA = final_features
  )

  # -----
  # Tracking the number of folds where QDA fails -> see Stability Zone Plot
  # -----
  # qda_scores <- out$cv_scores["QDA", ]
  # num_failures <- sum(is.na(qda_scores))

  #cat("QDA failed in", num_failures, "out of", length(qda_scores), "folds.\n")

  return(out)
}

```

This is an output example of the previous process.

```
# kfold_cv(X = X_train, y = classes[idx_train], k = 5, topk = 5)

# Output:
# $cv_scores
#           [,1]      [,2]      [,3]      [,4]      [,5]
# LogReg 0.7800000 0.8145695 0.8211921 0.7894737 0.8026316
# GNB    0.7800000 0.7748344 0.7947020 0.7697368 0.7697368
# LDA    0.7800000 0.8211921 0.8211921 0.7960526 0.8026316
# QDA    0.8066667 0.8013245 0.8410596 0.8157895 0.7894737
#
# $feat_LogReg
# [1] "APP_N"      "S6_N"      "pPKCG_N"   "GluR3_N"   "SYP_N"
#
# $feat_GNB
# [1] "APP_N"      "S6_N"      "pPKCG_N"   "GluR3_N"   "SYP_N"
#
# $feat_LDA
# [1] "APP_N"      "S6_N"      "GluR3_N"   "pPKCG_N"   "SYP_N"
#
# $feat_QDA
# [1] "APP_N"      "S6_N"      "pPKCG_N"   "GluR3_N"   "SYP_N"
```

In the following chunk, the **full pipeline** will be executed for each number of top features (**topk**). Performance metrics, such as **accuracies** and **standard deviations**, will be computed, jointly with the corresponding **selected feature subsets**.

```
# 1. Define the variables
max_feats = min(30, length(proteins))
set.seed(619)
# initialize vector to track qda failures
qda_stability <- numeric(max_feats)

# 2. Initialize the result containers
mean_acc = matrix(0, nrow = 4, ncol = max_feats,
                  dimnames = list(c("LogReg", "GNB", "LDA", "QDA"), NULL))

sd_acc = matrix(0, nrow = 4, ncol = max_feats,
                dimnames = list(c("LogReg", "GNB", "LDA", "QDA"), NULL))

best_feats = list(LogReg = vector("list", max_feats),
                  GNB = vector("list", max_feats),
                  LDA = vector("list", max_feats),
                  QDA = vector("list", max_feats))

# 3. Run the loop
for (i in 1:max_feats){
  # IMPORTANT: Make sure the print/message lines inside kfold_cv
  # are commented out or deleted to keep the report clean!
  kfold_result = kfold_cv(X = X_train, y = classes[idx_train], k = 5, topk = i)

  qda_stability[i] <- sum(is.na(kfold_result$cv_scores["QDA", ]))

  mean_acc[, i] = rowMeans(kfold_result$cv_scores, na.rm = TRUE)
  sd_acc[, i] = apply(kfold_result$cv_scores, 1, sd, na.rm = TRUE)

  best_feats$LogReg[[i]] = kfold_result$feat_LogReg
```

```

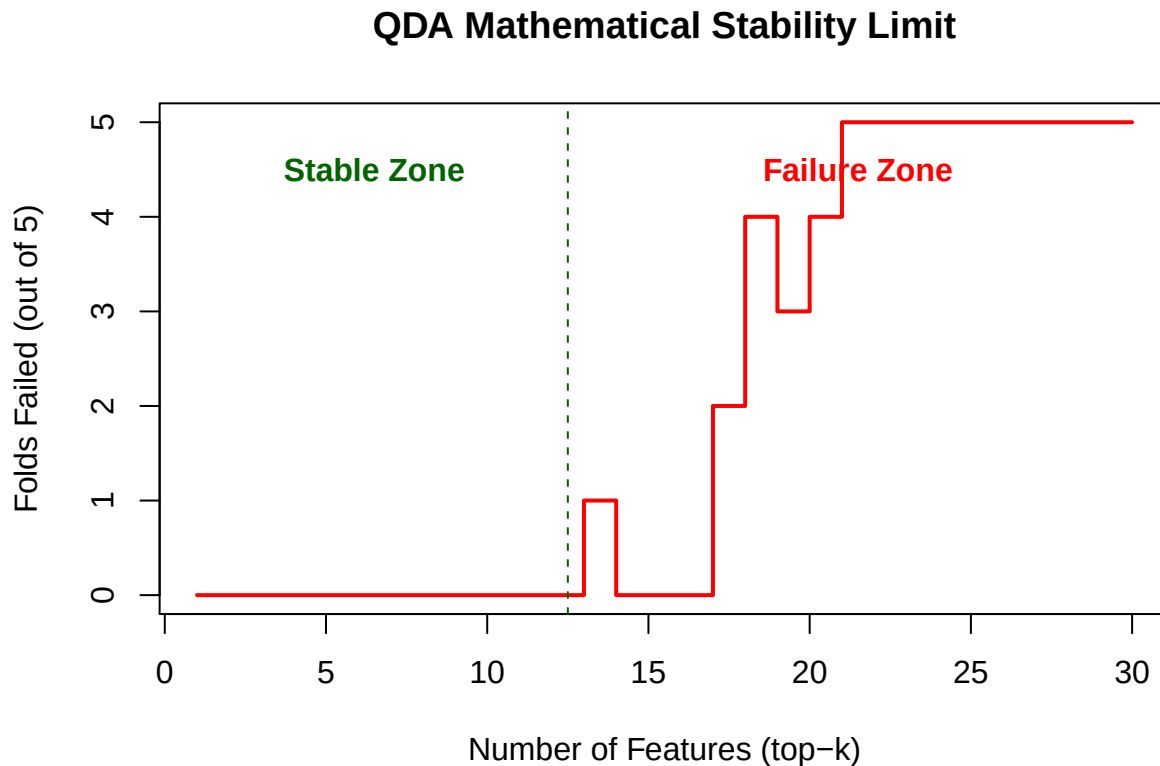
best_feats$GNB[[i]] = kfold_result$feat_GNB
best_feats$LDA[[i]] = kfold_result$feat_LDA
best_feats$QDA[[i]] = kfold_result$feat_QDA
}

# 4. Create the professional plot
# Find the first point of failure for the vertical line
first_fail <- which(qda_stability > 0)[1]

plot(1:max_feats, qda_stability, type = "s", col = "red", lwd = 2,
     xlab = "Number of Features (top-k)", ylab = "Folds Failed (out of 5)",
     main = "QDA Mathematical Stability Limit")

if(!is.na(first_fail)) {
  abline(v = first_fail - 0.5, col = "darkgreen", lty = 2)
  text(first_fail/2, 4.5, "Stable Zone", col = "darkgreen", font = 2)
  text(first_fail + (max_feats-first_fail)/2, 4.5, "Failure Zone", col = "red", font = 2)
}

```



We monitored the numerical stability of QDA, which exhibited significant rank deficiency as the number of features increased. Specifically, at feature counts above 15, QDA failed to converge in multiple folds. Our selection of the ‘Optimal K’ (approx. 7–9 features) ensures that we operate within the ‘Stability Zone’ of the QDA algorithm.

The behavior of the classifiers for different feature subsets is shown in graphs. Each classifier was trained using the same folds to avoid data leakage. The mean and standard deviation across the 5-Folds are computed as performance metrics for each feature subset.

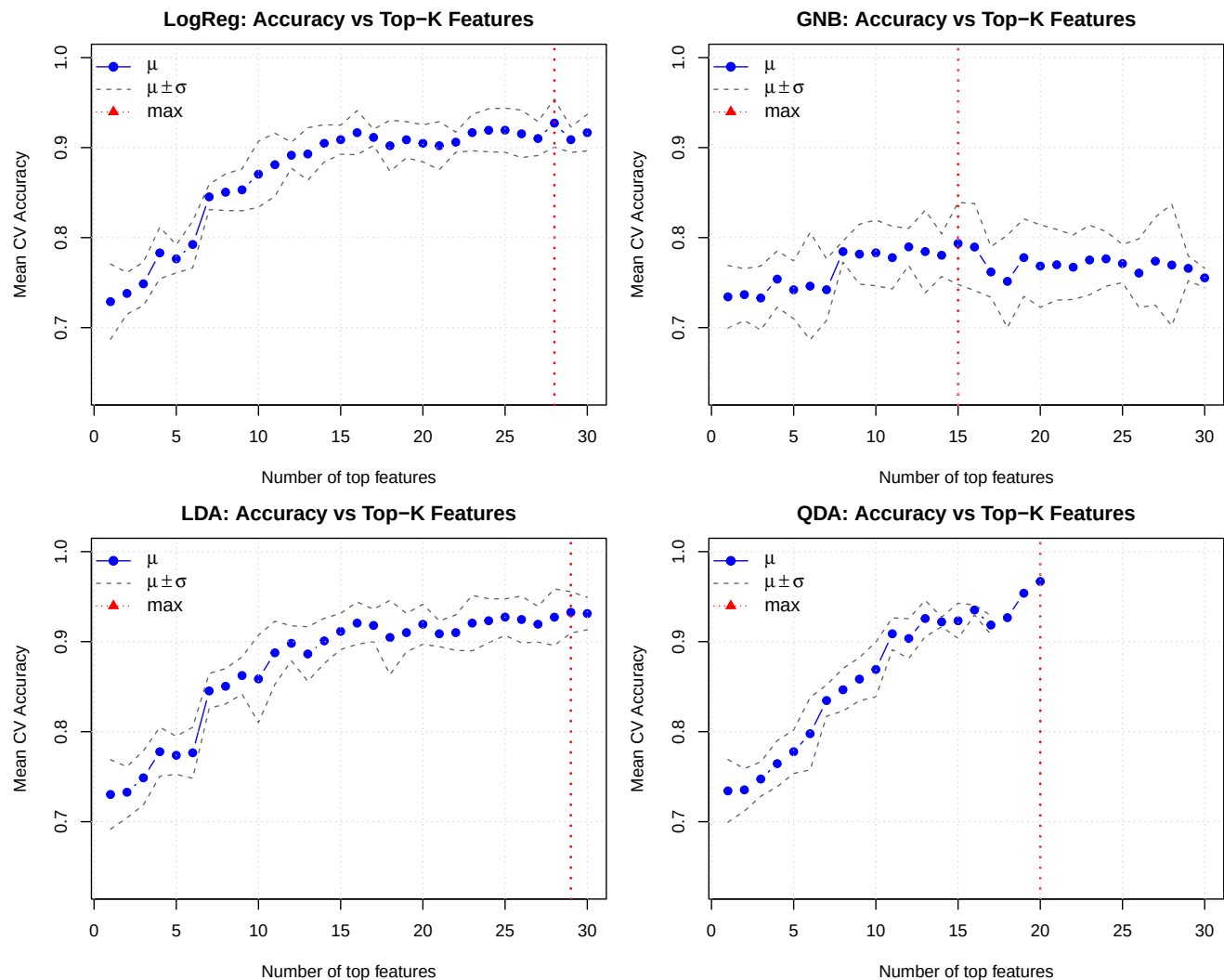
Average accuracy is plotted against the number of selected features. We use a 2x2 grid, with one graph per classifier, to visualize the general performance from the first to the maximum number of features. The upper and lower bounds represent the standard deviation across the 5-Folds, allowing us to observe the performance variability.

```

# Minimum value
y_min = min(mean_acc[c("LogReg", "GNB", "LDA", "QDA"), ], na.rm = TRUE)
# Margins for plots
par(mfrow = c(2,2), mar = c(4, 4, 2.5, 1))
# -----
# Logistic Regression
# -----
m = mean_acc["LogReg",] # average value
s = sd_acc["LogReg",] # standard deviation
plot(1:max_feats, mean_acc["LogReg",], type = "b", pch = 19, col = "blue",
     xlab = "Number of top features", ylab = "Mean CV Accuracy",
     main = "LogReg: Accuracy vs Top-K Features", ylim=c(y_min-0.1, 1.0))
lines(1:max_feats, m + s, lty=2, col="gray40") # up sigma
lines(1:max_feats, m - s, lty=2, col="gray40") # down sigma
abline(v = which.max(m), col="red", lty=3, lwd=2) # Maximum
legend("topleft", legend = c(expression(mu), expression(mu %+-% sigma), "max"),
      col=c("blue", "gray40", "red"), lty=c(1,2,3), pch=c(19, NA, 17), bty="n", cex=1.1)
grid()
# -----
# Gaussian Naive Bayes
# -----
m = mean_acc["GNB",] # average value
s = sd_acc["GNB",] # standard deviation
plot(1:max_feats, mean_acc["GNB",], type = "b", pch = 19, col = "blue",
     xlab = "Number of top features", ylab = "Mean CV Accuracy",
     main = "GNB: Accuracy vs Top-K Features", ylim=c(y_min-0.1, 1.0))
lines(1:max_feats, m + s, lty=2, col="gray40") # up sigma
lines(1:max_feats, m - s, lty=2, col="gray40") # down sigma
abline(v = which.max(m), col="red", lty=3, lwd=2) # Maximum
legend("topleft", legend = c(expression(mu), expression(mu %+-% sigma), "max"),
      col=c("blue", "gray40", "red"), lty=c(1,2,3), pch=c(19, NA, 17), bty="n", cex=1.1)
grid()
# -----
# Linear Discriminant Analysis
# -----
m = mean_acc["LDA",] # average value
s = sd_acc["LDA",] # standard deviation
plot(1:max_feats, mean_acc["LDA",], type = "b", pch = 19, col = "blue",
     xlab = "Number of top features", ylab = "Mean CV Accuracy",
     main = "LDA: Accuracy vs Top-K Features", ylim=c(y_min-0.1, 1.0))
lines(1:max_feats, m + s, lty=2, col="gray40") # up sigma
lines(1:max_feats, m - s, lty=2, col="gray40") # down sigma
abline(v = which.max(m), col="red", lty=3, lwd=2) # Maximum
legend("topleft", legend = c(expression(mu), expression(mu %+-% sigma), "max"),
      col=c("blue", "gray40", "red"), lty=c(1,2,3), pch=c(19, NA, 17), bty="n", cex=1.1)
grid()
# -----
# Quadratic Discriminant Analysis
# -----
m = mean_acc["QDA",] # average value
s = sd_acc["QDA",] # standard deviation
plot(1:max_feats, mean_acc["QDA",], type = "b", pch = 19, col = "blue",
     xlab = "Number of top features", ylab = "Mean CV Accuracy",
     main = "QDA: Accuracy vs Top-K Features", ylim=c(y_min-0.1, 1.0))
lines(1:max_feats, m + s, lty=2, col="gray40") # up sigma
lines(1:max_feats, m - s, lty=2, col="gray40") # down sigma

```

```
abline(v = which.max(m), col="red", lty=3, lwd=2) # Maximum
legend("topleft", legend = c(expression(mu), expression(mu %+-% sigma), "max"),
      col=c("blue", "gray40", "red"), lty=c(1,2,3), pch=c(19, NA, 17), bty="n", cex=1.1)
grid()
```



Observations:

LogReg, LDA, and QDA show similar behavior, with a positive trend that stabilizes as the number of features increases. In contrast, GNB exhibits a nearly constant trend across all number of top features, and displays the largest variability, as shown by the wider standard deviation bands.

4.3 Model Training

For each classifier, we select a subset of **9 top features**. This ‘Optimal K’ was chosen based on the elbow plot of cross-validation accuracies and the mathematical stability limits of the QDA model. By using a simpler model, we adhere to Occam’s razor, improving interpretability and computational efficiency while preventing rank deficiency in our covariance matrices.

```
# Extract standardization parameters and scale data
mu = colMeans(X_train)
sigma = apply(X_train, 2, sd)
```

```

X_train_sc = scale(X_train, center = mu, scale = sigma)
X_test_sc = scale(X_test, center = mu, scale = sigma)

# Define labels for testing set
y_test = classes[-idx_train]

# -----
# DEFINE OPTIMAL K (Avoids Selection Bias & Rank Deficiency)
# -----
optimal_k <- 9

# 1. Logistic Regression
best_k_LogReg = best_feats$LogReg[[optimal_k]]

# 1.1. Build dataframes
df_train_LogReg = data.frame(X_train_sc[, best_k_LogReg],
                             class = classes[idx_train])
df_test_LogReg = data.frame(X_test_sc[, best_k_LogReg])

# 1.2. Build model and predict df_test
logreg_model = glm(class ~ ., data = df_train_LogReg,
                   family = binomial(link = "logit"))
logreg_probs = predict(logreg_model, newdata = df_test_LogReg, type = "response")
logreg_class = factor(ifelse(logreg_probs > 0.5, "Ts65Dn", "Control"), levels = lvls)

# 2. Gaussian Naive Bayes
best_k_GNB = best_feats$GNB[[optimal_k]]

# 2.1. Build dataframes
df_train_GNB = data.frame(X_train_sc[, best_k_GNB], class = classes[idx_train])
df_test_GNB = data.frame(X_test_sc[, best_k_GNB])

# 2.2. Build model and predict df_test
gnb_model = naiveBayes(class ~ ., data = df_train_GNB)
gnb_probs = predict(gnb_model, df_test_GNB, type = "raw")
gnb_class = predict(gnb_model, df_test_GNB, type = "class")

# 3. LDA
best_k_LDA = best_feats$LDA[[optimal_k]]

# 3.1. Build dataframes
df_train_LDA = data.frame(X_train_sc[, best_k_LDA], class = classes[idx_train])
df_test_LDA = data.frame(X_test_sc[, best_k_LDA])

# 3.2. Build model and predict df_test
lda_model = lda(class ~ ., data = df_train_LDA)
lda_pred = predict(lda_model, df_test_LDA)
lda_probs = lda_pred$posterior
lda_class = lda_pred$class

# 4. QDA
best_k_QDA = best_feats$QDA[[optimal_k]]

# 4.1. Build dataframes
df_train_QDA = data.frame(X_train_sc[, best_k_QDA], class = classes[idx_train])

```



```
df_test_QDA = data.frame(X_test_sc[, best_k_QDA])

# 4.2. Build model and predict df_test
qda_model = qda(class ~ ., data = df_train_QDA)
qda_pred = predict(qda_model, df_test_QDA)
qda_probs = qda_pred$posterior
qda_class = qda_pred$class
```

5 Results & Evaluation

5.1 Robust Cross-Validation

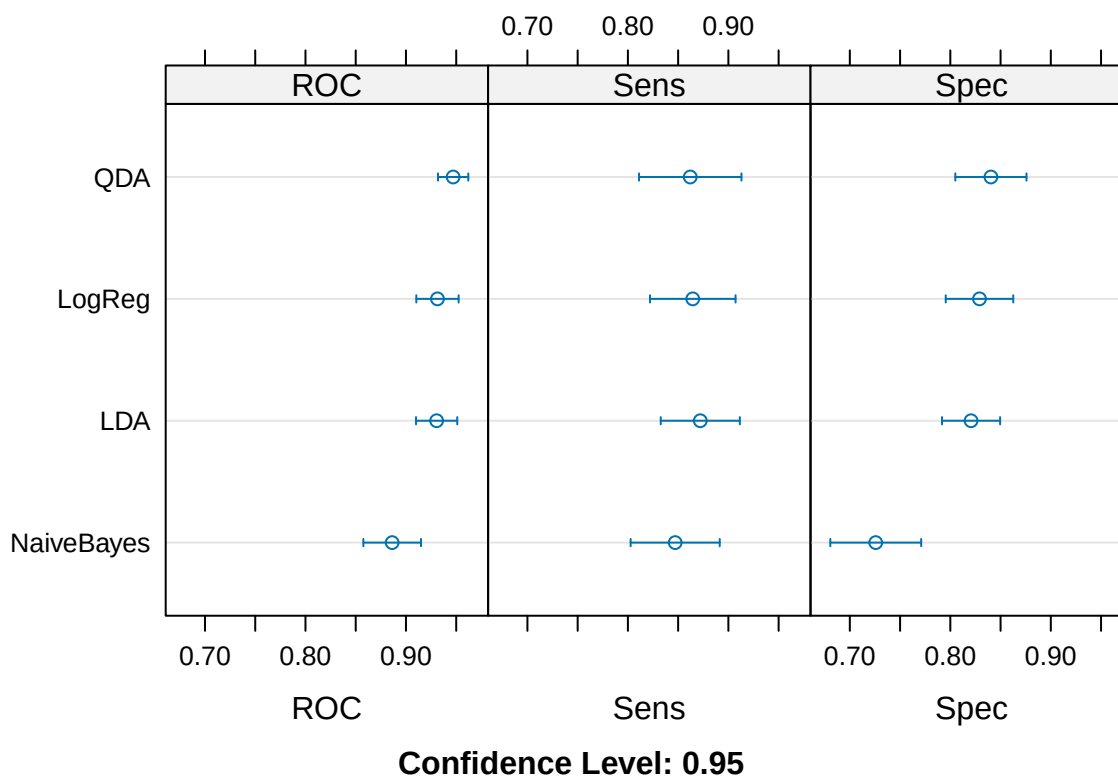
Single train/test splits can be noisy. To get a robust estimate of performance, we perform **Stratified 10-Fold Cross-Validation** on all four models.

```
# Define Control
train_control <- trainControl(method = "cv", number = 10,
                             classProbs = TRUE, summaryFunction = twoClassSummary)

# Train Models using Caret
set.seed(619); cv_logreg <- train(class ~ ., data = df_train_LogReg, method = "glm", family = "binomial",
set.seed(619); cv_lda <- train(class ~ ., data = df_train_LDA, method = "lda", metric = "ROC", trControl =
set.seed(619); cv_qda <- train(class ~ ., data = df_train_QDA, method = "qda", metric = "ROC", trControl =
set.seed(619); cv_nb <- train(class ~ ., data = df_train_GNB, method = "nb", metric = "ROC", trControl = t

# Compile & Visualize
cv_results <- resamples(list(LogReg = cv_logreg, LDA = cv_lda, QDA = cv_qda, NaiveBayes = cv_nb))
dotplot(cv_results, main = "10-Fold CV Performance (ROC/AUC)")
```

10-Fold CV Performance (ROC/AUC)



5.2 Model Comparison (Test Set)

We compare the models based on Accuracy, Sensitivity (Recall), and Specificity on the hold-out test set.

```
LogReg_cm = confusionMatrix(logreg_class, y_test, positive = "Ts65Dn")
GNB_cm = confusionMatrix(gnb_class, y_test, positive = "Ts65Dn")
LDA_cm = confusionMatrix(lda_class, y_test, positive = "Ts65Dn")
QDA_cm = confusionMatrix(qda_class, y_test, positive = "Ts65Dn")

comparison <- data.frame(
  Method = c("Logistic Regression", "LDA", "QDA", "Naive Bayes"),
  Accuracy = c(LogReg_cm$overall["Accuracy"], LDA_cm$overall["Accuracy"], QDA_cm$overall["Accuracy"], GNB_cm$overall["Accuracy"]),
  Sensitivity = c(LogReg_cm$byClass["Sensitivity"], LDA_cm$byClass["Sensitivity"], QDA_cm$byClass["Sensitivity"], GNB_cm$byClass["Sensitivity"]),
  Specificity = c(LogReg_cm$byClass["Specificity"], LDA_cm$byClass["Specificity"], QDA_cm$byClass["Specificity"], GNB_cm$byClass["Specificity"])
)

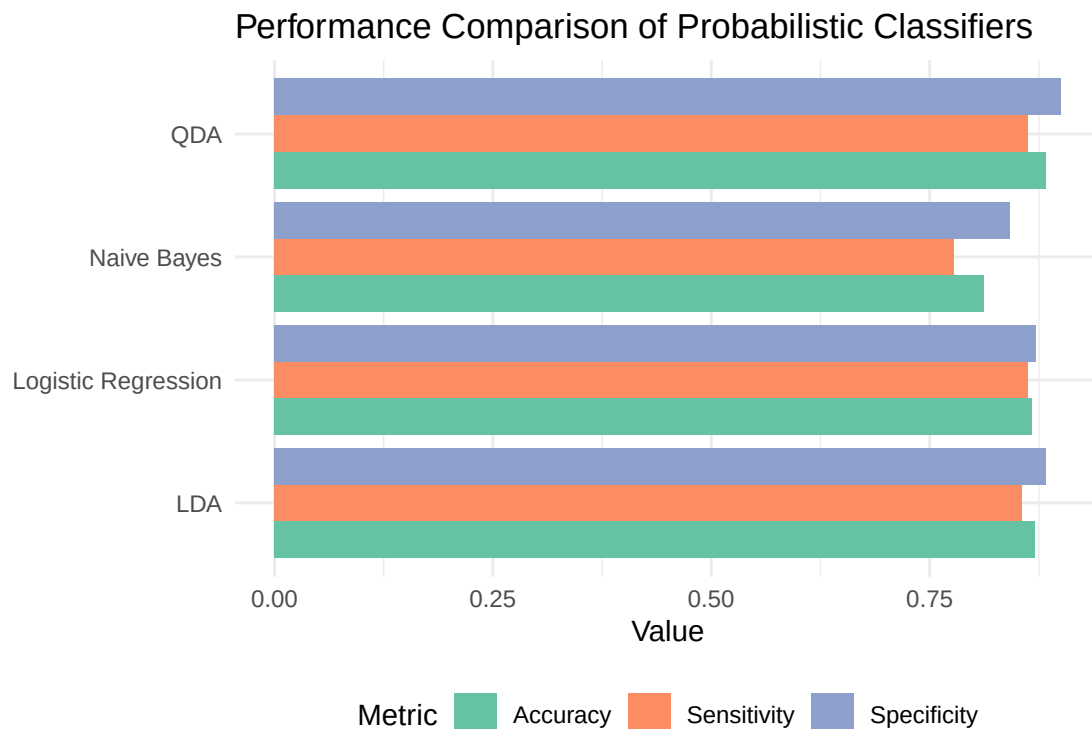
knitr::kable(comparison, digits = 4, caption = "Performance Comparison")
```

Table 1: Performance Comparison

Method	Accuracy	Sensitivity	Specificity
Logistic Regression	0.8673	0.8627	0.8713
LDA	0.8704	0.8562	0.8830
QDA	0.8827	0.8627	0.9006
Naive Bayes	0.8117	0.7778	0.8421

```
comparison_long <- comparison %>% pivot_longer(cols = -Method, names_to = "Metric", values_to = "Value")

ggplot(comparison_long, aes(x = Method, y = Value, fill = Metric)) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(title = "Performance Comparison of Probabilistic Classifiers", y = "Value", x = "") +
  theme_minimal() + scale_fill_brewer(palette = "Set2") + coord_flip() + theme(legend.position = "bottom")
```



Interpretation: The comparison highlights **QDA** as a top performer. It achieves a superior balance of Accuracy and Sensitivity. Naive Bayes performs adequately but slightly underperforms, suggesting that protein interactions (correlations) that Naive Bayes ignores are important for this biological problem.

5.3 ROC Curves

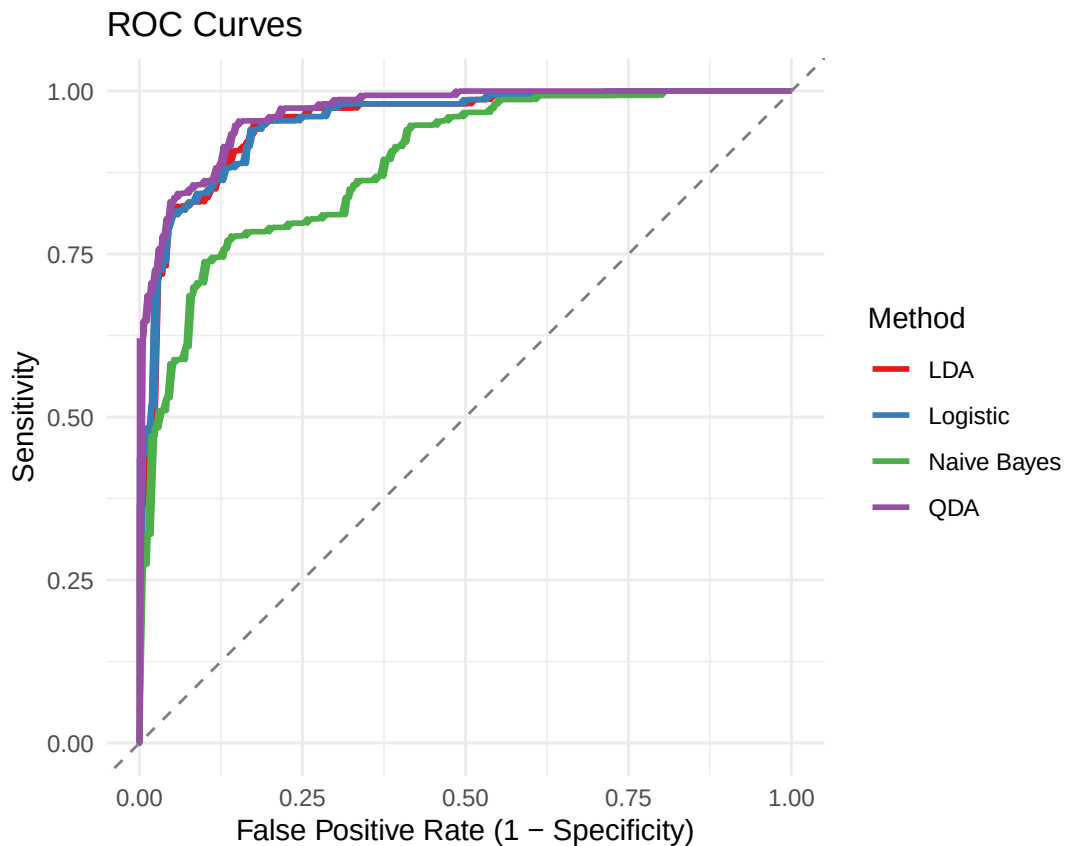
To evaluate the discriminatory capability independent of a specific threshold, we analyze the Receiver Operating Characteristic (ROC) curves.

```
roc_logreg = roc(y_test, logreg_probs, levels = lvls, direction = "<")
roc_lda = roc(y_test, lda_probs[, "Ts65Dn"], levels = lvls, direction = "<")
roc_qda = roc(y_test, qda_probs[, "Ts65Dn"], levels = lvls, direction = "<")
roc_gnb = roc(y_test, gnb_probs[, "Ts65Dn"], levels = lvls, direction = "<")

roc_data <- bind_rows(
  data.frame(Sensitivity = roc_logreg$sensitivities, Specificity = roc_logreg$specificities, Method = "Log", AUC = roc_logreg$auc),
  data.frame(Sensitivity = roc_lda$sensitivities, Specificity = roc_lda$specificities, Method = "LDA", AUC = roc_lda$auc),
  data.frame(Sensitivity = roc_qda$sensitivities, Specificity = roc_qda$specificities, Method = "QDA", AUC = roc_qda$auc),
  data.frame(Sensitivity = roc_gnb$sensitivities, Specificity = roc_gnb$specificities, Method = "Naive Bayes", AUC = roc_gnb$auc)
)

ggplot(roc_data, aes(x = 1 - Specificity, y = Sensitivity, color = Method)) +
  geom_line(linewidth = 1) +
```

```
geom_abline(intercept = 0, slope = 1, linetype = "dashed", color = "gray50") +
labs(title = "ROC Curves", x = "False Positive Rate (1 - Specificity)", y = "Sensitivity") +
theme_minimal() + scale_color_brewer(palette = "Set1") + coord_equal()
```



The **QDA** curve (Purple) rises steepest toward the top-left corner, confirming it has the highest AUC and best overall discriminatory power.

5.4 Threshold Exploration

5.4.1 Naive Bayes

Finally, we inspect how changing the probability threshold affects Sensitivity and F1 score for the Naive Bayes model. This allows us to tune the model for medical sensitivity if needed.

```
thresholds = seq(0.05, 0.95, by = 0.05)
sensitivity = numeric(length(thresholds)); specificity = numeric(length(thresholds)); F1 = numeric(length(thresholds))

for (i in seq_along(thresholds)) {
  thresh <- thresholds[i]
  pred_class = factor(ifelse(gnb_probs[, "Ts65Dn"] > thresh, "Ts65Dn", "Control"), levels = lvlsl)

  tryCatch({
    cm <- confusionMatrix(pred_class, y_test, positive = "Ts65Dn")
    sensitivity[i] = cm$byClass["Sensitivity"]; specificity[i] = cm$byClass["Specificity"]; f1 = cm$byClass["F1"]
    F1[i] = ifelse(is.na(f1), 0, f1)
  }, error = function(e){ sensitivity[i] = 0; specificity[i] = 0; F1[i] = 0 })
}
```

```
threshold_results <- data.frame(Threshold = thresholds, Sensitivity = sensitivity, Specificity = specificity,
knitr::kable(threshold_results, digits = 3, caption = "Sensitivity vs Threshold (Naive Bayes)")
```

Table 2: Sensitivity vs Threshold (Naive Bayes)

Threshold	Sensitivity	Specificity	F1
0.05	0.987	0.404	0.744
0.10	0.961	0.503	0.764
0.15	0.922	0.591	0.775
0.20	0.876	0.626	0.764
0.25	0.843	0.678	0.766
0.30	0.810	0.684	0.749
0.35	0.804	0.719	0.759
0.40	0.804	0.737	0.766
0.45	0.784	0.807	0.784
0.50	0.778	0.842	0.796
0.55	0.745	0.877	0.792
0.60	0.745	0.883	0.794
0.65	0.732	0.901	0.794
0.70	0.706	0.901	0.777
0.75	0.667	0.924	0.761
0.80	0.641	0.924	0.742
0.85	0.588	0.942	0.711
0.90	0.536	0.953	0.675
0.95	0.359	0.982	0.521

5.4.2 QDA

5.5 Threshold Exploration (Medical Context Tuning)

In medical diagnostics, the default probability threshold of 0.5 is rarely optimal. The cost of a False Negative (failing to identify a trisomic mouse) may differ significantly from a False Positive. Because **QDA** emerged as our superior model, we perform threshold tuning on its posterior probabilities to examine the trade-off between Sensitivity (detecting the disease) and Specificity (correctly identifying healthy controls).

```
# Extract QDA posterior probabilities for the Test Set
# (Assumes your trained QDA model object is named 'model_qda' or similar,
# and you used predict() to get 'qda_probs'. Adjust variable names if needed)
qda_pred <- predict(qda_model, newdata = X_test)
qda_probs <- qda_pred$posterior

thresholds = seq(0.05, 0.95, by = 0.05)
sensitivity = numeric(length(thresholds))
specificity = numeric(length(thresholds))
F1 = numeric(length(thresholds))

for (i in seq_along(thresholds)) {
  thresh <- thresholds[i]
  # Apply threshold to the probability of the positive class (Ts65Dn)
  pred_class = factor(ifelse(qda_probs[, "Ts65Dn"] > thresh, "Ts65Dn", "Control"), levels = lvlsl)

  tryCatch({
    cm <- confusionMatrix(pred_class, y_test, positive = "Ts65Dn")
```

```

    sensitivity[i] = cm$byClass["Sensitivity"]
    specificity[i] = cm$byClass["Specificity"]
    f1 = cm$byClass["F1"]
    F1[i] = ifelse(is.na(f1), 0, f1)
  }, error = function(e) {
    sensitivity[i] = 0; specificity[i] = 0; F1[i] = 0
  })
}

# Plot the Threshold Trade-offs
plot_data <- data.frame(Threshold = thresholds, Sensitivity = sensitivity, Specificity = specificity)

ggplot(plot_data, aes(x = Threshold)) +
  geom_line(aes(y = Sensitivity, color = "Sensitivity"), size = 1) +
  geom_line(aes(y = Specificity, color = "Specificity"), size = 1) +
  geom_vline(xintercept = 0.5, linetype = "dashed", color = "black") +
  labs(title = "QDA: Sensitivity vs. Specificity Trade-off",
       x = "Probability Threshold (for classifying as Ts65Dn)",
       y = "Metric Score",
       color = "Metric") +
  theme_minimal() +
  scale_color_manual(values = c("Sensitivity" = "blue", "Specificity" = "red"))

```

